Ders 9 Genişletilmiş Notu: Sınav Değerlendirmesi ve Makale Sunumları

Ders 9 Genişletilmiş Notu: Sınav Değerlendirmesi ve Makale Sunumları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Proje Yönetiminin Aşamaları: Yol Haritası Sorusu
- 2. 1. Sunum: Çevik Projelerde Proje Yöneticisi Hâlâ Var mı?
- 3. 2. Sunum: Monte Carlo Simülasyonuyla Yazılım Projesi Zaman Yönetimi

1. Proje Yönetiminin Aşamaları: Yol Haritası Sorusu

Hoca derse, sınavda sorduğu yol haritası (roadmap) sorusunun cevabını anlatarak başladı. Soru şuydu: Bir proje yönetiminde hangi adımlar vardır, yani proje yönetimi sırasında hangi aşamalardan geçeriz? Bir öğrenci cevabı e-postayla sormuştu. Hoca da herkes duysun diye konuyu derste açıkladı.

Hocanın verdiği cevapta aşamalar şu sırayla ilerler:

1. **Hazırlık:** Projeye hazırlık, yani projeyi anlama (understanding) ve tanımlama (defining) aşamaları.
2. **Planlama:** Projenin planlanması.
3. **Yürütme ve izleme:** Projenin yürütülmesi ve izlenmesi.
4. **Kapanış:** Projenin kapatılması ve projeyle ilgili birikimin oluşturulması.

Hoca üçüncü aşamayı yazılım geliştirme örneğiyle somutlaştırdı. Yazılım geliştirme işinde ilerlerken belli noktalarda, genellikle kilometre taşlarında (milestone), şu soru sorulur: Planla eş güdümlü gidiyor muyuz? Cevaba göre ya plan revize edilir ya da plana uygun gidilmeye çalışılır.

> **Hocanın vurgusu:** Bu soru proje yönetiminin aşamalarını soruyor. Proje planının içindeki adımları yazmak bu sorunun cevabı değildir. Hoca bu konudan derste söz ettiğini de hatırlattı.
>
> **Ders sonrası not:** Altyazıda kapanış aşaması için “projeyle ilişkili navha oluşturmak” yazıyor. Bu sözcük büyük olasılıkla “know-how” olarak söylendi. Yani kapanışta projeden edinilen birikim kayda geçirilir.

2. 1. Sunum: Çevik Projelerde Proje Yöneticisi Hâlâ Var mı?

2.1 Makale

Bir öğrenci “Does the Project Manager Still Exist in Agile Software Development Projects?” adlı makaleyi sundu. Makale Yeni Zelanda’daki bir üniversiteden üç araştırmacı tarafından yazılmış ve 2016’da 23. Asya Pasifik Yazılım Mühendisliği Konferansı’nda yayımlanmış.

Ders sonrası not: Altyazıda üniversitenin adı “Otland” olarak geçiyor. Bu büyük olasılıkla Auckland Üniversitesi’dir. Yazarların kurumu makalenin kendisinden kontrol edilmedi.

2.2 Arka Plan: Çevik Proje Yönetimi ve Scrum

Çevik (agile) proje yönetimi değişime açık, artımlı ve yinelemeli bir proje yönetimi yaklaşımıdır. Temelleri 2001’de bir grup yazılım geliştiricinin yayımladığı çevik yazılım geliştirme manifestosuyla atıldı ve gelişmeye devam ediyor. Kendi kendini örgütleyen takımlar ve müşteri iş birliği gibi ilkelerle değişen iş gereksinimlerine hızlı yanıt vermeyi hedefler.

Çevik yaklaşım tek bir yöntem değildir, birçok metodolojiyi içinde barındırır. Sunumda dinamik yazılım geliştirme yöntemi, özellik güdümlü geliştirme (feature driven development) ve uç programlama (extreme programming, XP) örnek verildi. En çok tercih edilenlerden biri Scrum’dır. Öğrencinin aktardığı son sektör araştırmalarına göre katılımcıların %75’i kuruluşlarında Scrum veya Scrum hibriti modeller kullanıyor.

Scrum’a özgü etkinlikler ve öğeler olarak şunlar sayıldı:

- Kullanıcı öyküleri (user stories)
- Ürün biriktirme listesi (product backlog)
- Günlük Scrum
- Sprint inceleme toplantısı
- Sprint planlama toplantısı

2.3 Proje Yöneticisi Kimdir?

Proje yöneticisi geleneksel takımlarda ve çevik takımlarda farklı konumlarda bulunur. Slaytta bunu gösteren bir görsel vardı. Şelale (waterfall) gibi geleneksel yöntemlerde proje yöneticisi daha merkezî bir rol oynar ve yapı daha hiyerarşiktir.

Genel olarak bir proje yöneticisinden liderlik, ekip oluşturma, motivasyon, iletişim kurma, etkileme, karar verme, planlama ve koçluk gibi özellikler beklenir. Öğrenciye göre bu beklentiler çevik takımlar için de geçerlidir. Fark, karar verme yetkisinin dağılımındadır. Çevik takımlarda bu yetki ekip ile yönetici arasında dengeli biçimde dağılır. Takımlar kendi görevlerini kendileri seçer ve daha çok kendileri yerine getirir.

2.4 Scrum ve XP’de Roller

Scrum ve XP gibi çevik yöntemlerde proje yöneticisi iş unvanı teoride kaldırılmıştır. Scrum metodolojisinde bu adla bir rol yer almaz. Onun yerine iki rol vardır:

- **Ürün sahibi (Product Owner):** Müşteriyi temsil eder. Ekibi müşteri gereksinimleri doğrultusunda çalışmaya yönlendirir. Ürün özelliklerinin önceliklendirilmesinden sorumludur.
- **Scrum ustası (Scrum Master):** Ekibin işleyişini kolaylaştırır ve önündeki engelleri kaldırır.

XP’de koç, danışman, izci (tracker), programcı, müşteri, test eden ve büyük patron rolleri yer alır. Bunlar arasında koç, sorumlulukları açısından proje yöneticisinin geleneksel tanımına benzer. Koçun sorumlulukları sürecin sahipliğini ve takıma rehberlik etmeyi gerektirir.

2.5 Araştırma Sorusu ve Yöntem

Makalenin çıkış sorusu şudur: Proje yöneticisi unvanı teoride kaldırıldı, peki sektördeki gerçek kullanımda da durum böyle mi?

Bunun için Google Forms üzerinden, çevrim içi platformlarda bir anket yapılmış. Ankete 31 farklı ülkeden 97 kişi katılmış. Sorular iki bölümden oluşuyor:

- **Demografik sorular:** İkamet edilen ülke, projenin sektörü, yaş grubu, sektördeki toplam iş tecrübesi, çevik ve geleneksel projelerdeki iş deneyimi.

- **Çalışma konusuna ilişkin sorular:** İş unvanı, mevcut projede kullanılan yazılım geliştirme metodolojisi, ekip üye sayısı, projede proje yöneticisi olup olmadığı, yoksa ekibi yönetmekten kimin sorumlu olduğu, ekibin aynı yerde mi yoksa dağıtık mı çalıştığı.

2.6 Demografik Sonuçlar

Aşağıdaki tablolar sunumda sözlü olarak aktarılan değerleri toplar.

Ülke	Oran
ABD	%29
Hindistan	%13
Hollanda	%5
Kanada, Almanya, Birleşik Krallık	%4'er
Diğer (İsveç, Yeni Zelanda, Avustralya, Rusya, Romanya, Meksika vb., 25 ülke)	%41 (ara adım eklendi)

Diğer kategorisinin oranı sunumda söylenmedi: $100 - (29 + 13 + 5 + 3 \cdot 4) = 41$. Ülke sayısı da tutuyor: Adı geçen 6 ülke ile 25 ülke toplamı 31 eder.

Sektör	Oran
Finans ve bankacılık	%17
Bilgi teknolojileri	%15
Sağlık hizmetleri	%13
Telekomünikasyon	%7
Yazılım	%6
Diğer (eğlence, eğitim, satış vb.)	%37

Bu oranların toplamı %95'tir. Öğrenci sektör belirtmeyenlerden de söz etti, ama oranı altyazıda geçmiyor.

Tecrübe (yıl)	Toplam iş tecrübesi	Çevik projelerde	Geleneksel projelerde
0-5	%10	%65	%30
5-10	%12	%29	%29
10-20	%44	11-15 ve 16-20 dilimlerinde %4 ve %1	daha düşük oranlar
20 üstü	%33		

Toplam tecrübe ve çevik tecrübe sütunlarının toplamı %99'dur. Bu fark büyük olasılıkla yuvarlamadan gelir. Tabloya göre katılımcıların çoğu sektörde kıdemlidir, ama çevik projelerdeki tecrübeleri çoğunlukla 5 yılın altındadır.

Ekip büyüklüğü	Oran
0-5 kişi	%13
5-10 kişi	%39
10-15 kişi	%23
15-20 ve 20-25 kişi	%4 ve %5
25 üstü	%15

İş unvanlarında katılımcıların %24'ü proje yöneticisi, %11'i Scrum Master, %4'ü çevik koç olduğunu belirtmiş. Kalan %61 (ara adım eklendi) CEO, ürün sahibi ya da test ve kalite güvence gibi farklı unvanlar bildirmiş.

2.7 Asıl Bulgular

Projede proje yöneticisi var mı? Katılımcıların %67'si evet, %33'ü hayır demiş. Yani unvan teoride kalkmış olsa da projelerin büyük çoğunluğunda hâlâ bir proje yöneticisi var.

Ekip bir arada mı çalışıyor? Ekiplerin %55'i dağıtık, %45'i bir arada çalışıyor. Bu soru ile proje yöneticisinin varlığı arasında bağlantı bulunmuş. Dağıtık çalışan projelerde proje yöneticisinin bulunma yüzdesi daha yüksek. Bir arada çalışan ekiplerde ise proje yöneticisinin görülmeme oranı daha yüksek.

> **Ders sonrası not:** Öğrenci önce "%55'i uzaktan çalışmıyor" dedi, hemen ardından "yani takım bir arada çalışmıyor" diye açıkladı. Devamındaki anlatıma göre doğrusu: %55 dağıtık, %45 bir arada.

Hangi metodoloji kullanılıyor? Dağılım şöyledir:

Kullanım	Oran	Ayrıntı
Tek çevik yöntem	%58	Scrum %49, Kanban %2; özellik güdümlü geliştirme ve XP gibi diğerleri genelde %1 civarı
İki yöntemli hibrit	%19	En yüksek Scrum + Kanban %8; ardından Scrum + özellik güdümlü geliştirme, Scrum + şelale, Scrum + diğer çevik yöntemler
İkiden fazla yöntemli hibrit	%23	En üstte Scrum + Kanban + XP, ardından Scrum + Kanban + şelale

Üç oranın toplamı %100'dür. Hibritlerin sayılan örneklerinde Scrum her seferinde yer alıyor, bazılarında şelaleyle birlikte.

Ekip büyüklüğü ile proje yöneticisinin varlığı. Araştırmada en belirgin çıkan ilişki budur. Slayttaki tabloya göre öğrenci şunları aktardı:

- Proje yöneticisi olmadığının en çok belirtildiği grup 5-10 kişilik ekipler.
- 25 ve üstü ekiplerde proje yöneticisinin bulunma olasılığı çok yüksek.
- 10-15 kişilik ekiplerde de bulunma durumu diğer büyüklüklere göre daha yüksek.
- 0-5 kişilik küçük ekiplerde proje yöneticisi daha çok bulunmayabiliyor.

> **Ders sonrası not:** Öğrenci 5-10 kişilik ekipleri önce "proje yöneticisi olmadığının en çok belirtildiği grup" olarak andı, sonuçta ise bu ekiplerde yöneticinin bulunma ihtimalinin 5'ten küçük ekiplerden çok daha yüksek olduğunu söyledi. Bu grup anketin en kalabalık grubu olduğu için (%39) ilk ifade büyük olasılıkla mutlak kişi sayısını anlatıyor, oranı değil. Tablo görülmeden kesinleştirilemez.

2.8 Sonuçların Yorumu

Küçük ekipler ve Scrum'un ideal takım büyüklüğü. 5-10 kişilik ekiplerde proje yöneticisinin bulunma ihtimali, 5'ten küçük ekiplerden çok daha yüksek. Öğrenci bunu literatürdeki bilgiyle karşılaştırdı: Scrum için ideal takım büyüklüğü 3 ile 9 üye arasındadır. Bu büyüklükteki takımlarda proje yöneticisine yine de ihtiyaç duyulmuş ya da böyle bir kişi yer almış. Ayrıca 3-10 üyeli ekipleri bildiren birkaç ampirik çalışmada da proje yöneticisi rolü sıkça görülmüş.

> **Ders sonrası not:** Altyazıda bu bulgunun Scrum'un ideal takım büyüklüğü bilgisiyle "gelişmekte" olduğu yazıyor. Cümlenin mantığına göre sözcük büyük olasılıkla "çelişmekte"dir. Scrum teoride bu büyüklükte yönetici öngörmezken, ankette yönetici bulunuyor.

Olası neden. Öğrenciye göre bu gözlemin nedenlerinden biri kurumsal yapıdır. Birçok kurum ekip düzeyinde çevik yöntemler kullansa da ekip seviyesinin üzerinde hâlâ geleneksel proje yönetimi metodolojilerini sürdürüyor. Bu da proje yöneticisinin bulunma oranını yükseltebilir.

Büyük ekipler. 20 kişiden büyük ekipler üzerinde yapılan ampirik çalışmalarda da proje yöneticisinin varlığı kaydediliyor. Sunumda Martin'in 20 üyeli bir ekip üzerine çalışması anıldı. Bu çalışmada proje yöneticisi, uzmanların projeye katılımını teşvik ederek ekibin işleyişini kolaylaştırmış. Anket sonucu da bu bulguyu destekliyor: Büyük ekiplerde proje yöneticisinin bulunma olasılığı küçük ekiplere göre daha yüksek.

Dağıtık ekipler. Dağıtık ekiplerde proje yöneticisi yüksek oranda yer alıyor. Öğrenciye göre bunun nedeni, dağıtık ekiplerin koordinasyon gerektirmesi olabilir. Bu durum başka çalışmalarda da kaydedilmiş.

Sunumun sonunda soru gelmedi.

3. 2. Sunum: Monte Carlo Simülasyonuyla Yazılım Projesi Zaman Yönetimi

İkinci sunumu yapan öğrenci, Monte Carlo simülasyonuna dayalı yazılım projesi zaman yönetimi araştırması üzerine bir referans makaleyi ve birkaç ek kaynağı inceledi. Sunumun sırası şöyleydi: zaman yönetimi, Monte Carlo yöntemi, PERT ağları, önerilen adımlar ve makaledeki örnek durum simülasyonu.

3.1 Motivasyon

Yazılım sektörü büyüdükçe rekabet artıyor. Bu rekabet içinde zamanlaması ve planlaması kötü yapılan projelerin başarı oranı düşüyor. Bu durum birçok şirket üzerinde, örneğin girişimlerde, ekonomik baskı yaratıyor. Bu nedenle proje yönetimi ve onun içinde proje zaman yönetimi giderek önem kazanıyor. Referans makale, zamanlama yönetiminin doğruluğunu artırmak ve projenin başarıyla kabul edilmesini sağlamak için Monte Carlo yönteminin kullanılmasını öneriyor.

3.2 Proje Zaman Yönetimi

Yazılım projesinin zaman yönetimi; projenin amacına ulaşmak, çıktısını üretmek ve kapsam planında öngörülen işi tamamlamak için, proje kapsamı belirlendikten sonra yürütülen bir proje yönetimi faaliyetidir. Projeyi zamanında tamamlayabilmek için görevlerin süreci ve sorunları kontrol edilerek ilerlenir. Sunan öğrenci, sektörde çalışırken bu konudaki eksikliklerin sıkıntısını kendisinin de yaşadığını söyleyerek zaman yönetimini yazılım proje yönetiminin en önemli parçalarından biri olarak gördüğünü belirtti.

Yazılım projelerinde her aşamanın süresi belirsizdir ve çeşitli faktörlerden etkilenir. Sunumda şu örnekler verildi: Ekibin bu tür yazılımda deneyimi olup olmaması, kurumda o yazılım türüyle ilgili bir birikim (know-how) bulunup bulunmaması ve daha önceden hesaplanıp kaydedilmiş bir metrik olup olmaması. Burada insan kaynaklı hatalara yol açabilecek durumlar da vardır.

3.3 Monte Carlo Yöntemi

Monte Carlo yöntemi, stokastik özelliklere sahip problemleri tekrarlanan istatistiksel deneylerle çözmeye dayanır. Bir çeşit dijital simülasyon deneyidir. Çözülecek problem bir olayın meydana gelme olasılığı veya rastgele bir değişkenin beklenen değeri ise, olayın meydana gelme sıklığı veya değişkenin ortalama değeri deneysel yöntemle elde edilir ve bu değerler gerçek değerler olarak kullanılabilir.

> **Ders sonrası not:** Öğrenci yöntemin 1940 yılında Stanislav Ulam tarafından bulunduğunu söyledi. Yöntem büyük olasılıkla tam 1940'ta değil, 1940'ların ortasında ortaya çıktı. Altyazıdaki "hüküter bilim insanı" ifadesi anlaşılmıyor; Ulam büyük olasılıkla bilgisayar bilimcisi değil matematikçi olarak tanımlanmalıdır.

Olasılık ve rastgele değişken fikri, yöntemin yazılım zamanlamasına neden uygulanabileceğini açıklıyor. Makaleye göre sistemin karmaşıklığı ve projenin doğrusal olmayan süreci nedeniyle proje süresini tahmin edecek doğru bir matematiksel model kurmak zordur. Bu yüzden şu hipotez ortaya konuyor: Monte Carlo simülasyonu, risklerin zaman ve maliyet öngörüsü üzerindeki etkilerini analiz etmek için yazılım zaman yönetiminde kullanılabilir.

Makalenin yöntem için öne sürdüğü özellikler şunlardır:

- Daha güvenilir tahmin değerlerine yaklaşmayı sağlar. Simülasyon sayısı arttıkça tahminler daha doğru olur ve grafik daha düzgün bir eğriye gider.
- Proje yöneticisinin deneyimi ve tercihi gibi öznel faktörlere bağımlılığı düşüktür. Öznel müdahaleyi önleyerek süre tahmininin nesnelliğini ve bilimselliğini artırır.
- Analizle projenin belirli bir süre içinde tamamlanma olasılığı ve tamamlanma güvenilirliği elde edilebilir.

3.4 PERT Tekniğinin Hatırlatılması

Öğrenci, derste görülen PERT tekniğini Monte Carlo ile birleşeceği için kısaca hatırlattı. PERT, geçmişe dayalı veri olmadığında, özellikle büyük ve karmaşık projelerde zaman çizelgelerini belirlemek ve süre tahmini yapmak için kullanılan istatistiksel bir yöntemdir. Zaman faktörü maliyet faktöründen daha önemli olduğunda tercih edilir. Benzer proje deneyimi olmadan bir yazılım projesinin süresini tahmin etmeyi kolaylaştırır.

PERT, projeyi elverişli, elverişsiz ve olağan çalışma koşullarında tamamlamak için gereken süreleri dikkate alır ve nihai tahmini ağırlıklı ortalamayla elde eder. Öğrenciye göre bu şekilde elde edilen tahmin daha bilimsel ve güvenilirdir.

> **Ek bilgi (derste anlatılmadı):** PERT'in olağan formülleri, iyimser süre t_o, en olası süre t_m ve kötümser süre t_p için $t_e = \dfrac{t_o + 4t_m + t_p}{6}$ ve $\sigma = \dfrac{t_p - t_o}{6}$ biçimindedir. Slayttaki formüller büyük olasılıkla bunlardır.

3.5 Monte Carlo Yönteminin Önerilen Adımları

Makalenin yazılım proje yönetimine uygulama için önerdiği adımlar şunlardır:

1. Proje görevlere ayrılır. Her görev için üç süre tahmini yapılır: iyimser, kötümser ve en olası.
2. Standart sapma ve t_e değeri hesaplanır. Bunlar uygun bir olasılık dağılımı modelini seçmek için kullanılır.
3. Monte Carlo simülasyonuyla olasılık modeli kurulur. Bu model proje süresinin farklı olasılıklar altındaki dağılımını simüle eder. Olasılık dağılımına göre rastgele sayılarla çok sayıda tekrarlı hesap yapılır ve sonuca yakınsanır.
4. Hesaplanan süre değerleri üzerinde veri işleme yapılır. Böylece en küçük değer, en büyük değer, matematiksel beklenen değer, standart sapma, her günün olasılığı ve kritik yol olasılığı elde edilir.

Referans makalede rastgele sayılar Python ile üretilmiş. Öğrenciye göre en büyük avantaj şudur: Geçmiş proje deneyimi olmasa bile, üç süre tahminine dayanarak sözde rastgele (pseudo-random) sayılar üretilebilir. Simülasyon bu sayılarla koşulduğunda gerçek ve bilimsel veriye yaklaşmak kolaylaşır.

> **Ders sonrası not:** Öğrenci "rastgele sayılar yerine yüksek benzerliğe sahip sözde rastgele sayılar" dedi. Bilgisayarda üretilen rastgele sayılar büyük olasılıkla zaten sözde rastgeledir. Bu ayrı bir seçenek değil, olağan durumdur.

3.6 Örnek Durum: İnsan Kaynakları Yönetim Sistemi

Makale, küçük çaplı bir insan kaynakları yönetim sistemi yazılımı üzerinde çalışmış. Proje altı adıma ayrılmış: proje planlama, talep (gereksinim) analizi, tasarım, kodlama, test ve kabul. Her adımın iyimser, kötümser ve en olası süresi bir tabloda verilmiş. Bu tablodan PERT diyagramı ve kritik yol çıkarılmış.

> **Ek bilgi (derste anlatılmadı):** Kritik yol, başlangıçtan bitişe en uzun süren etkinlik dizisidir. Proje süresini bu yol belirler. Süreler rastgele olduğunda her simülasyon denemesinde en uzun yol değişebilir. Bu yüzden simülasyon, her yolun kritik yol olma olasılığını da verir.

Öğrenci iki aday kritik yol aktardı:

Yol	İyimser	Kötümser	Ortalama
Yol 1: A-B-C-D-F	112 gün	176 gün	144 gün (“146-144” diye söylendi)
Yol 2: A-B-C-F	92 gün	142 gün	117 gün

İki ortalama, iyimser ve kötümser değerin tam orta noktasıyla aynıdır: $(112 + 176)/2 = 144$ ve $(92 + 142)/2 = 117$ (ara adım eklendi).

> **Ders sonrası not:** İkinci yol altyazıda A-B-C-F olarak geçiyor, ama bu hâliyle tutarsızdır. A-B-C-F, A-B-C-D-F’nin içinde kaldığı için hiçbir zaman ondan uzun olamaz ve kritik yol olma olasılığı sıfır olurdu. Öğrenci ise birazdan bu yolun düşük ihtimalle kritik olabileceğini ve E etkinliğinin göz ardı edilmemesi gerektiğini söyledi. Bu nedenle ikinci yol büyük olasılıkla A-B-C-E-F’dir.

3.7 Kaç Simülasyon Koşulmalı?

Monte Carlo’da simülasyon sayısı önemlidir. Ne kadar çok simülasyon koşulursa sonuçlar o kadar kararlı olur. Ama “ne kadar çok o kadar iyi” diye gereğinden fazla koşmak süreyi uzatır ve zarara dönüşür. Bu yüzden gerekli simülasyon sayısını ölçmek için bir formül verilmiş. Formül en iyi örneklem büyüklüğünü, projenin standart sapmasını ve izin verilen hatayı içeriyor.

Öğrenci değerleri Excel’de hesaplayınca $n = 12.845$ bulmuş. Makalede ise 25.000 simülasyon koşulmuş. Öğrencinin vurguladığı nokta şudur: Önemli olan tam 25.000 seçmek değil, hesaplanan n değerinden fazla simülasyon koşmaktır. Burada $25.000 > 12.845$ olduğu için koşul sağlanıyor. Bu da daha kararlı ve gerçeğe yakın değerlere ulaşmak için gereklidir.

> **Ek bilgi (derste anlatılmadı):** Bu tür örneklem büyüklüğü hesapları genelde $n = \left(\frac{z\,\sigma}{E}\right)^2$ biçimindedir. Burada σ standart sapma, E izin verilen hata, z istenen güven düzeyine karşılık gelen katsayıdır. Makaledeki formül büyük olasılıkla buna benzerdir, ama altyazıdan doğrulanmıyor.

3.8 Simülasyon Sonuçları

Simülasyonda beklenen proje süresi 174 gün olarak aktarıldı. Standart sapma, en büyük ve en küçük süre de slaytta gösterildi. Sonuçlara göre projenin kritik yolu %97,71 olasılıkla Yol 1, yani A-B-C-D-F çıktı. İki aday yol olduğuna göre diğer yolun kritik olma olasılığı $100 - 97,71 = 2,29$ yüzdedir (ara adım eklendi).

> **Ders sonrası not:** 174 günlük beklenen süre, verilen yol değerleriyle tutarlı görünmüyor. Yol 1 en çok 176 gün sürüyor ve ortalaması 144 gün. PERT formülüyle $t_e = 174$ olması için $t_m = (6 \cdot 174 - 112 - 176)/4 = 189$ gerekirdi. Bu değer 176 gün üst sınırının üzerindedir. 174 değeri büyük olasılıkla altyazı veya aktarım hatasıdır. Doğru değer makaleden kontrol edilmelidir.

Öğrenci, sonucu okurken şu uyarıyı yaptı: İkinci yolun kritik olma ihtimali düşük olsa da proje fiilen uygulanırken bilinmeyen durumlarla ve risk analizinde belirlenen risklerle karşılaşılabilir. Bu yüzden E etkinliği, yani test, kritik yolda değil diye göz ardı edilmemelidir. Öğrenci, sonucun test için böyle çıkmasını dramatik bulduğunu da söyledi. Kendi gözlemine göre test, sektörde en çok göz ardı edilen kısımdır. Oysa müşteriye teslimden sonraki kaliteyi güvence altına alan aşama testtir.

Simülasyon sayısının etkisi bir grafikle gösterildi. Simülasyon sayısı arttıkça eğri düzgünleşiyor ve sonuçlar sabit bir eğilime doğru gidiyor.

3.9 Dikkat Edilecek Noktalar ve Faydalar

Yöntemin dikkat isteyen tarafları şunlardır:

- **Başlangıç değerleri gerekir:** Simülasyon için üç süre tahmininin yapılmış olması şarttır.

- **Tahmin kalitesi sonucu belirler:** Yöntem kişisel hatalardan uzaklaşmayı amaçlasa da tahminlerin kalitesi kilit rol oynar. Sözde rastgele sayıların dayandığı referans değerleri sonuçta proje yöneticisi ve ekip belirler. Öğrenci bunu açık bir soru işareti olarak gördü ve üzerinde daha fazla çalışılması gerektiğini düşündü.
- **Bütün etkinlikler dâhil edilmelidir:** Hesap tek bir etkinlik için değil, tüm etkinlikler için yapılmalıdır.
- **Risk değerlendirmesi tamamlanmış olmalıdır:** Karşılaşılabilecek riskler göz önünde bulundurulmalıdır.

Öne sürülen faydalar ise şunlardır:

- Proje risklerini zaman açısından değerlendirmeye yardım eder.
- Zaman ve maliyet aşımlarını ve hataları görmeyi sağlar.
- Gerçekçi bütçe ve zaman çizelgesi hazırlamayı sağlar.
- Gerçekçi zaman yönetimi ve riskler görülebildiği için risk yönetimine yönetim desteği almayı kolaylaştırır.
- Gerçekçi verilerle doğru karar vermeyi destekler.
- Makalenin iddiasına göre kilometre taşlarına ulaşma ve proje teslim tarihini tutturma olasılığını artırır.

3.10 Sonuç ve Gecikmenin Teste Binmesi

Yazılım proje yönetimi boyunca, proje kaynaklı veya kişisel birçok belirsizlik zamanlama tahminlerinin yanlış yapılmasına yol açıyor. Karmaşıklık arttıkça proje süresini doğru tahmin etmek sektör genelinde ciddi bir problem hâline geliyor. Makale, Monte Carlo tekniğinin süre tahminini son duruma daha yakın hâle getirdiğini, tahminin güvenilirliğini ve proje yönetiminin verimliliğini artırdığını iddia ediyor. Böylece süre yönetimi için yeni bir yönetim fikri sunuyor.

Öğrenci sonuca sektör gözlemini ekledi. Zaman yönetiminin kaliteyle bağı şöyle kuruluyor:

1. Gereksinim analizi sırasında bir zaman aşımı olur. Çizelge iyi belirlenmediyse bu aşım göz ardı edilir.
2. Geliştirme sırasında da aynı şey olur. Gecikmeler birbirinin üzerine biner.
3. Son gün değişmez, çünkü müşteriye teslim ya da kabul testine çıkış tarihi sabittir.
4. Gecikme çoğunlukla teste doğru ötelenir. Test paketi geç geldiği için test geç başlar, ama testin bitmesi gereken tarih aynı kalır.
5. Birçok hata gözden kaçar, metriklere ulaşılamaz ve projenin kalitesi ciddi biçimde etkilenir.

Öğrenci bu yüzden yazılım zaman yönetiminin ciddi biçimde iyileştirilmesini önemli buluyor ve Monte Carlo simülasyonunu bu konuda bir ışık olarak görüyor. Makaleyi seçme nedeni de bu. Yine de yöntemin eksikleri olduğunu ve üzerinde daha fazla çalışılması gerektiğini söyledi.

Dersin Toparlaması

Derste önce proje yönetiminin aşamaları netleşti: hazırlık (anlama ve tanımlama), planlama, yürütme ve izleme, kapanış. Bu aşamalar proje planının maddeleriyle karıştırılmamalıdır. İzleme aşamasında kilometre taşlarında planla karşılaştırma yapılır ve gerekirse plan revize edilir.

İki sunum da bu çerçevenin farklı yerlerine değdi. İlki, proje yöneticisi unvanı Scrum ve XP'de teoride kalkmış olsa da ankete katılanların %67'sinin projesinde hâlâ bir proje yöneticisi bulunduğunu, bu varlığın ekip büyüklüğü ve dağıtık çalışmayla ilişkili olduğunu gösterdi. İkincisi, PERT'in üç süre tahminini Monte Carlo simülasyonuyla birleştirerek süre dağılımı ve kritik yol olasılıkları elde etmeyi anlattı, ama sonucun yine tahminlerin kalitesine bağlı olduğunu da hatırlattı. Hoca iki sunumu da iyi buldu ve sonraki sunumlar için örnek olduklarını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.